THE STATE UNIVERSITY OF ZANZIBAR

Data Science Practical Training

Data Science Project Technical Report

From Problem Definition to Model Deployment

Field	Information
Project title	Unsupervised Visual Anomaly Detection and Localization on High-Resolution Industrial Components Using Deep Feature Reconstruction
Group members	JOSEPH GODFREY RITTE 24BCS008 LILIAN PETER MUSHI 25BCS023 JUMA MASSOUD JUMA 24BCS009 ABDULKADIR BAKAR MBWANA 24BCS028 IBRAHIM ABDALLA SHARIF 24BCS006
Internal Supervisor or Mentor	Rabia Said
Submission date	8.10.2026

Academic Year 2025/2026

Report Contents

Section	Title
1	Introduction
2	Dataset and Data Understanding
3	Data Preparation
4	Exploratory Data Analysis
5	Model Development and Evaluation
6	Results and Interpretation
7	Application Development
8	System Testing and Deployment
9	Ethical Considerations and Limitations
10	Conclusion and Recommendations
	References and Appendices

1 Introduction

1.1 Background

Quality inspection is a critical step in high-precision manufacturing. Small surface defects such as scratches, dents, contamination, deformations and structural deviations must be found on components before final assembly. Manual visual inspection is slow, costly and subjective, and it is affected by fatigue. Automated visual inspection using machine learning is therefore attractive, but defective examples are rare in real production and new defect types cannot be fully predicted in advance [1][2].

The people affected are quality-control engineers, assembly-line operators and manufacturing auditors, who need a fast, consistent first-pass decision on whether a component looks normal. The MVTec Anomaly Detection (MVTec AD) benchmark was created for exactly this situation: it provides defect-free training images, anomalous test images across several object and texture categories, and pixel-precise defect masks [1][2].

1.2 Problem Statement

High-precision manufacturing lines need to identify small and previously unseen surface defects before final assembly. A supervised classifier is difficult to maintain because labelled defective samples are scarce and unpredictable. Group 3 therefore proposed an unsupervised system that learns the appearance of defect-free components and flags test images whose local visual features differ from that normal pattern.

The gap addressed in this report is the step from raw benchmark images to a working, evaluated and deployed prototype. A data-driven solution is appropriate because the image evidence (colour, texture and edge structure) can be converted into numerical features, compared across models, and delivered to non-technical users through a web application. **This report documents the data preparation, a supervised baseline model and the Streamlit application that were completed, and states clearly (Section 10.1) which of the original unsupervised, deep-feature objectives remain to be implemented.**

1.3 Project Objectives

General objective. To build and test an unsupervised system that detects and localizes anomalies in MVTec AD images, classifies parts as normal or defective, and identifies anomalous regions without using defective images during training.

Specific objectives.

- Extract patch-level feature embeddings from normal high-resolution training images using a pretrained CNN backbone.
- Construct a distance-based memory-bank scoring method that produces image-level decisions and pixel-level anomaly heatmaps.
- Evaluate detection and localization performance against the provided ground-truth masks using Image-Level AUROC and Pixel-Level PRO metrics.

Practical objectives of the work reported here. (a) Prepare an analysis-ready dataset from the MVTec AD images; (b) develop and compare basic Machine Learning models as a supervised baseline; (c) deliver the best model in an easy-to-use Streamlit application. The achievement of each objective is assessed in Section 10.1.

1.4 Research or Project Questions

The analysis, model and application were designed to answer the following questions:

- Is the selected MVTec AD data suitable for the project, and what are its structure, quality and class balance?
- Can basic Machine Learning models using simple image-level features separate normal from anomalous images, and which model performs best?
- Which features contribute most to the prediction, and do they reflect visual defect evidence rather than object category?
- Can the selected model be delivered as a usable application with validated inputs and an understandable result?
- Can patch-level CNN embeddings with a memory bank detect and localize defects? (**Not yet answered**: this is the planned next stage.)

1.5 Scope of the Project

Dataset: the working copy of MVTec AD in the group's Google Drive: 3,260 images from 10 object and texture categories (one row per image in cleaned_project_dataset.csv). **Population:** industrial object and texture images photographed under the controlled conditions of the benchmark. **Variables:** the target image_label and 21 predictors (nine image features, width, height and ten one-hot category indicators).

Prediction task: image-level binary classification (0 = normal, 1 = anomalous) using a supervised baseline. **Application boundaries:** a locally run prototype that classifies one image (or one set of typed feature values) at a time, as decision support only. **Out of scope:** CNN patch embeddings, the memory-bank detector, pixel-level heatmaps, AUROC/PRO evaluation, user authentication and online deployment.

2 Dataset and Data Understanding

2.1 Dataset Source and Access

The data come from the MVTec Anomaly Detection dataset (MVTec AD) published by MVTec Software GmbH [1][2]. The images were obtained from the official benchmark and a Kaggle mirror [3] and stored in Google Drive; the original images are never modified. The dataset is released under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International licence (CC BY-NC-SA 4.0), so it may be used for non-commercial purposes with attribution and share-alike conditions.

2.2 Dataset Description

The working dataset has 3,260 observations (one image each) and eight metadata variables, summarised in Table 1. The full MVTec AD benchmark contains 5,354 images in 15 categories [2]; the working copy used here contains 10 categories. About 76% of the images are normal and 24% are anomalous.

Table 1. Overview of the dataset used in this project.

Dataset Element	Description
Dataset name	MVTec Anomaly Detection (MVTec AD), working copy
Source	MVTec Software GmbH, https://www.mvtec.com/research-teaching/datasets/mvtec-ad; Kaggle mirror https://www.kaggle.com/datasets/ipythonx/mvtec-ad
Observations	3,260 images (unit of analysis: one image); 2,477 normal and 783 anomalous
Variables	8 metadata columns in the cleaned file; 21 predictor columns after feature engineering
Target variable	image_label: 0 = normal ("good") image, 1 = image from a named defect folder (anomalous)
Problem type	Binary classification (supervised baseline)

2.3 Key Variables

Table 2 is the variable dictionary. The nine image features were created from the pixels of each image (Section 3.2); the original columns marked "excluded" were not used as predictors.

Table 2. Variable dictionary and role of each variable in the model.

Variable	Meaning	Type / values	Role
image_path	Location of the image file	Text (file path)	Identifier; excluded
category	Object or texture type	10 categories: bottle, carpet, grid, leather, metal_nut, pill, screw, tile, toothbrush, zipper	Predictor (one-hot encoded)
split	Original MVTec partition	train / test	Excluded (leakage: train holds only normal images)
defect_type	Type of defect, or "good"	33 values including "good"	Excluded (derived from the label)

Variable	Meaning	Type / values	Role
image_label	0 = normal, 1 = anomalous	Integer 0/1	Target
mask_path	Ground-truth defect mask	Text; NOT_APPLICABLE for normal images	Excluded; reserved for localization evaluation
width, height	Image size in pixels	Integer, 700 to 1,024 (images are square)	Predictors
mean_r, mean_g, mean_b	Average red, green and blue value of the 128×128 image	Decimal, 0 to 255	Predictors (engineered)
std_r, std_g, std_b	Spread of each colour channel	Decimal, 0 to 128	Predictors (engineered)
brightness	Average grayscale intensity	Decimal, 0 to 255	Predictor (engineered)
contrast	Standard deviation of grayscale intensity	Decimal, 0 to 128	Predictor (engineered)
edge_density	Share of pixels detected as edges (Canny)	Decimal, 0 to 1	Predictor (engineered)

2.4 Initial Data Quality Assessment

The initial audit of the raw metadata is summarised in Table 3. The only missing field was mask_path (2,477 of 3,260 rows), which is structurally expected because normal images have no defect mask. No duplicates were found, and all 3,260 images could be opened during feature extraction (0 failures). The most important quality issues for modelling are the class imbalance and the fact that the original training partition contains only normal images.

Table 3. Initial data-quality assessment.

Issue checked	Evidence	Action taken
Missing values	mask_path missing in 2,477 rows; all other columns complete	Coded as NOT_APPLICABLE; rows kept
Duplicate records	0 duplicate rows	None needed
Inconsistent types / text	Mixed object and integer types; possible whitespace	Types coerced; text stripped
Image sizes / outliers	Square images, constant within each category (700 to 1,024 px)	Retained; sizes are legitimate
Class distribution	2,477 normal (75.98%) and 783 anomalous (24.02%)	Stratified split; precision/recall/F1 used
Original split vs. class	train: 2,210 normal, 0 anomalous; test: 267 normal, 783 anomalous	New stratified split created
Category coverage	leather has 163 normal and 0 anomalous images	Noted as a limitation
Unreadable images	0 of 3,260 failed during feature extraction	None needed

3 Data Preparation

3.1 Data Cleaning

The metadata was cleaned on a copy of the raw table so that the original is preserved. Missing mask_path values were not imputed or dropped: an absent mask is meaningful for normal images, so the rows were kept and explicitly coded NOT_APPLICABLE, which avoids deleting 2,477 valid normal observations. Numeric columns (image_label, width, height) were converted to numeric types and text columns were trimmed of whitespace to keep category labels consistent. No duplicates existed, so no rows were removed, and image sizes were retained because different categories legitimately use different resolutions. After cleaning, the table still has 3,260 rows and 8 columns with 0 missing values, and it was saved as cleaned_project_dataset.csv. The original image files were never resized, edited or deleted.

3.2 Feature Selection and Engineering

Because the cleaned metadata contains no visual information, the predictors had to be computed from the images (Table 4). Each image was resized to 128×128 pixels and nine numerical features were calculated: the mean and standard deviation of the red, green and blue channels, the brightness (mean grayscale value), the contrast (standard deviation of grayscale) and the edge density (share of edge pixels found by the Canny detector). These are hand-crafted, image-level statistics; they are **not** CNN patch embeddings.

Table 4. Variables retained, removed and created for modelling.

Action	Variables	Reason
Removed	image_path, mask_path	Identifiers with no visual meaning
Removed	defect_type	Derived from the label ("good" = 0, any other value = 1); would leak the answer
Removed	split	The original train partition contains only normal images, so it carries label information
Retained	category, width, height	Known before inspection and not derived from the label
Created	mean_r/g/b, std_r/g/b, brightness, contrast, edge_density	Give the models genuine visual evidence from each image

The final predictor matrix has 3,260 rows and 21 columns (11 numeric columns and 10 category indicators).

3.3 Encoding and Scaling

The categorical variable category was one-hot encoded into ten indicator columns. The numeric columns were standardised with a StandardScaler that was **fitted on the training set only** and then applied to the test set, so that no test information leaked into preprocessing. Scaling was used for Logistic Regression, which is sensitive to feature scale; the Decision Tree and Random Forest were trained on unscaled features because tree splits are unaffected by monotonic scaling. The column list, the scaler and the model were saved so that the application reproduces exactly the same input format.

3.4 Data Splitting

The data were split once into a training set (80%) and a test set (20%) using stratified random sampling on image_label (random_state = 42). Stratification keeps the normal/anomalous proportions almost identical in both sets (76.0% normal in training and 75.9% in testing). The original MVTec split could not be used because its training partition has no anomalous examples. **No separate validation set was created.** The three models were compared on the test set, which is a limitation discussed in Sections 5.2 and 9.2.

Table 5. Data splitting.

Data Subset	Size and Purpose
Training set	2,608 images (80%); 24.0% anomalous; used to fit all models and the scaler
Validation set	Not used (no separate validation partition and no hyperparameter search)
Test set	652 images (20%; 495 normal and 157 anomalous); held out during training and used to compare and evaluate the models

4 Exploratory Data Analysis

4.1 Descriptive Statistics

Table 6 shows the frequency distribution by category and class. Every category has between 163 and 480 images, and the share of anomalous images ranges from 0% (leather) to 32.5% (pill). The images are all square, with sizes of 700 to 1,024 pixels (mean 931.4, standard deviation 116.9, median 1,024) and a constant size inside each category, so width and height mainly identify the category rather than the defect status. Numerical summaries of the nine engineered features were not exported from the Colab run (the optional preparation notebook exports them to model_features.csv); the first rows of the feature table are shown in the notebook output.

Table 6. Number of images per category and class.

Category	Normal	Anomalous	Total	% anomalous
bottle	229	63	292	21.6%
carpet	307	89	396	22.5%
grid	285	57	342	16.7%
leather	163	0	163	0.0%
metal_nut	242	93	335	27.8%
pill	293	141	434	32.5%
screw	361	119	480	24.8%
tile	253	72	325	22.2%
toothbrush	72	30	102	29.4%
zipper	272	119	391	30.4%
All categories	2,477	783	3,260	24.0%

4.2 Visual Analysis

Figure 1 confirms the class imbalance: about three images in four are normal, so a model that always answered "normal" would score about 76% accuracy while detecting no defect. For this reason precision, recall and F1 on the anomalous class are used. The right panel shows that all 783 anomalous images are in the original test partition and none are in the training partition.

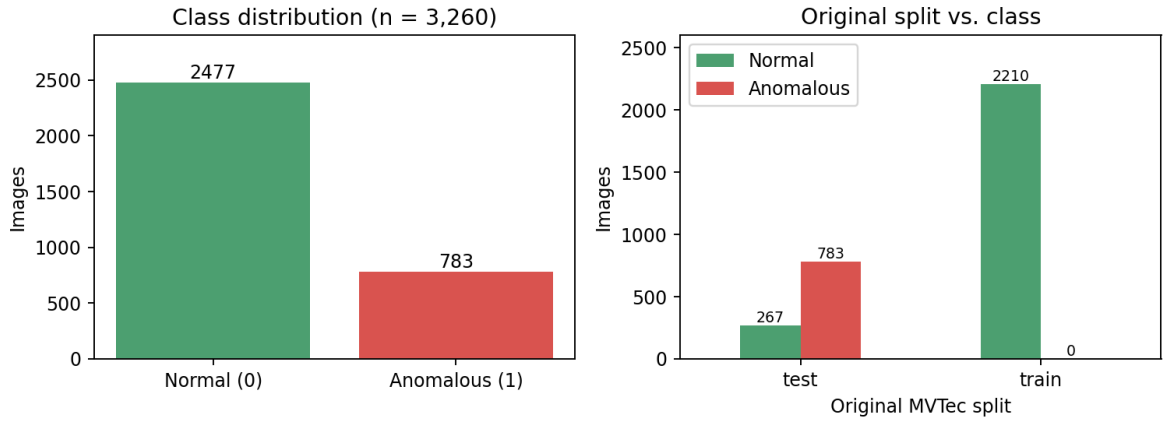


Figure 1. Class distribution and original MVTec split versus class.

Figure 2 shows that categories differ in size and in anomaly share, and that leather contains no anomalous images in this working copy. The model therefore cannot learn what a leather defect looks like, and results for that category cannot be assessed.

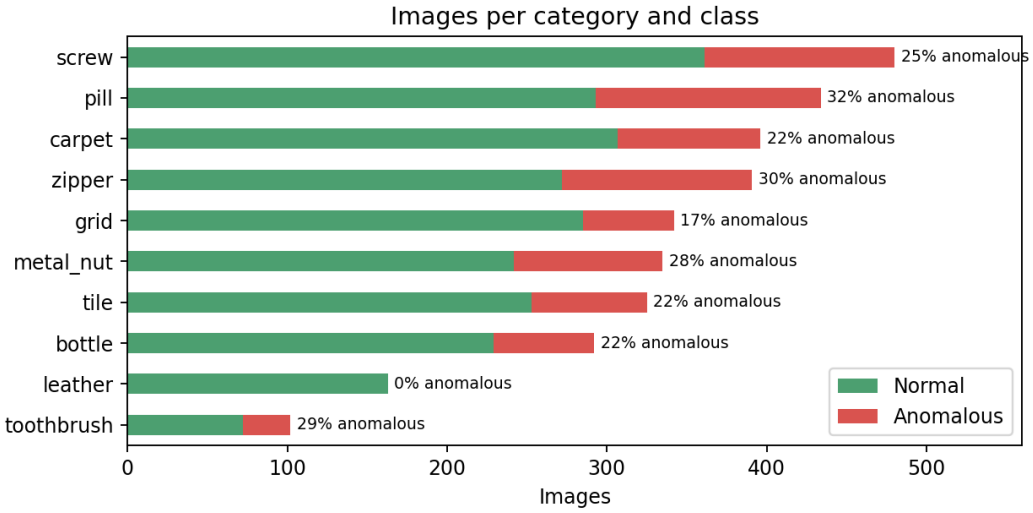


Figure 2. Images per category, split by class.

Figure 3 shows that anomalies are spread over many defect types, each with few examples (the commonest, "color", has 66 images). This supports the view that defect types cannot be anticipated one by one, which is the original motivation for an unsupervised approach.

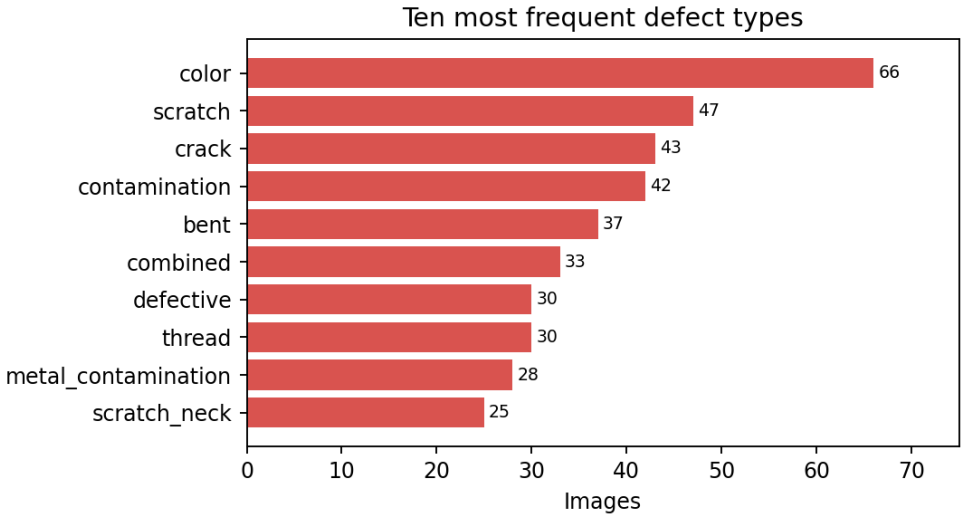


Figure 3. The ten most frequent defect types.

4.3 Key Findings from EDA

- The data are imbalanced (76% / 24%), so accuracy alone is misleading and stratified splitting is required.
- The original training partition is one-class (normal only), so it cannot train a supervised classifier; a new stratified split was needed.
- Categories differ in size, image resolution and anomaly share, and leather has no anomalous examples, which limits category-level conclusions.
- Defect types are numerous (32 types plus "good") and individually rare, motivating anomaly-detection methods that do not need labelled defects.
- defect_type and split are tied to the label and were excluded to prevent data leakage.

5 Model Development and Evaluation

5.1 Models Developed

Three classification models were trained so that the choice of the final model rests on evidence:

- **Logistic Regression** is a simple, interpretable linear baseline. It assumes the classes can be separated by a linear boundary in the (scaled) feature space.
- **Decision Tree** (maximum depth 8) captures non-linear rules and feature interactions and is easy to explain. It assumes the data can be separated by axis-aligned splits.
- **Random Forest** (300 trees) averages many decision trees trained on random subsets of rows and features, which reduces the overfitting of a single tree and handles correlated features such as the three colour channels [6].

All three models used balanced class weights so that the minority (anomalous) class is not ignored. These are supervised models: they were trained with labelled normal and anomalous images, so they form a baseline and not the unsupervised method named in the project title.

5.2 Model Training and Tuning

Each model was fitted on the training set (2,608 images) and used to predict the test set (652 images). Hyperparameters were fixed in advance and **no tuning or cross-validation was performed**: Logistic Regression (max_iter = 1000, balanced class weights), Decision Tree (max_depth = 8, balanced class weights) and Random Forest (n_estimators = 300, unlimited depth, balanced class weights); random_state was 42 for reproducibility. Because there was no validation set, the final model was chosen by its F1-score on the test set. This is a methodological weakness: the reported test performance of the selected model may be slightly optimistic, and a validation set or cross-validation should be used in future work.

5.3 Evaluation Measures

The measures are computed for the anomalous class, which is the class of interest. **Accuracy** is the share of all images classified correctly. **Precision** is the share of images flagged as anomalous that really are anomalous (it falls when there are many false alarms). **Recall** is the share of truly anomalous images that were found (it falls when defects are missed). **F1-score** is the harmonic mean of precision and recall and was used to choose the model. The **confusion matrix** shows the counts of correct and incorrect decisions. Because the data are imbalanced, accuracy alone is not sufficient. ROC-AUC was not computed in this stage; the AUROC and PRO measures required by the specific objectives will be used for the unsupervised detector.

5.4 Model Comparison

Table 7 gives the results on the 652-image test set (no validation results exist because no validation set was used).

Table 7. Model comparison on the test set (anomalous class unless stated).

Model	Accuracy	Precision	Recall	F1-score
Random Forest	0.853	0.729	0.618	0.669
Decision Tree	0.715	0.442	0.707	0.544
Logistic Regression	0.601	0.352	0.783	0.486

Random Forest has the highest accuracy, precision and F1-score. Logistic Regression finds the most anomalies (recall 0.783) but also flags many normal images (precision 0.352), which in practice would send many good parts to manual inspection. The Decision Tree lies between the two. The per-class results are given in Appendix B.

6 Results and Interpretation

6.1 Selected Model

Random Forest was selected because it has the highest F1-score on the anomalous class (0.669), the highest accuracy (0.853) and the highest precision (0.729), and it produces by far the fewest false alarms (36 of 495 normal images). It is less interpretable than Logistic Regression but provides a feature-importance ranking. Its main weakness is recall: it found 97 of 157 anomalous images (61.8%). If missing a defect is judged more costly than a false alarm, the decision threshold (0.5 in the application) could be lowered. The stability of the result was not assessed because only one split was used and the model was selected on the test set.

6.2 Interpretation of Predictions

Figure 4 ranks the ten most important predictors of the Random Forest. edge_density is the strongest (0.133), followed by the blue-channel statistics std_b (0.119) and mean_b (0.118), then mean_r (0.113) and mean_g (0.105); brightness, std_g, std_r and contrast contribute similar, moderate amounts (0.090 to 0.099). Width contributes very little (0.006), and every category indicator and height contributes less than width.

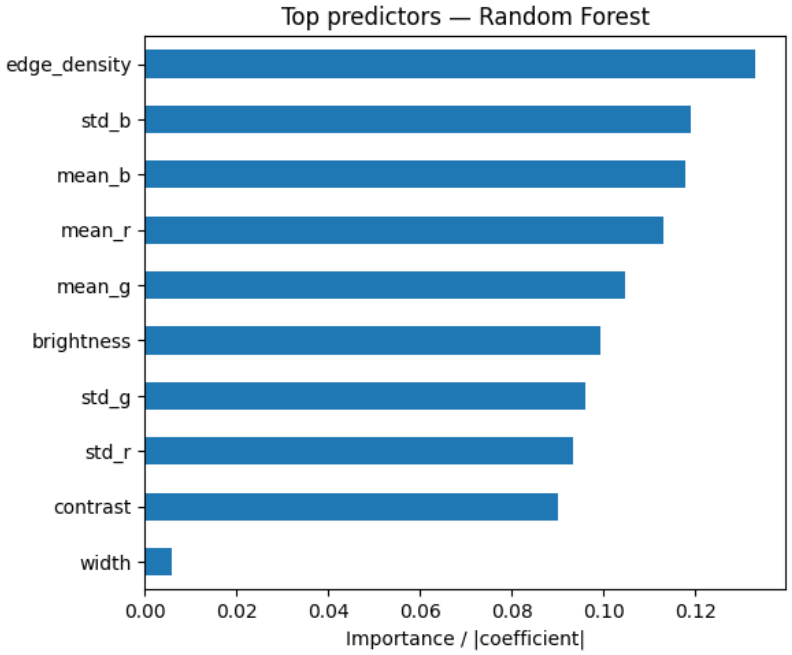


Figure 4. Top ten predictors of the selected Random Forest.

In plain language, the model mainly looks at how many edges an image contains and at its colour balance, and flags images whose edge and colour pattern resembles the defective examples it saw in training. That edge density leads is consistent with scratches, cracks, holes and broken parts disturbing local edge patterns, and the colour features are consistent with contamination and colour defects. Because no category indicator is among the top predictors, the model does not appear to rely mainly on the object type. Importance values describe how much the model used a feature; they do not prove that the feature causes anomalies.

Example: in the application, three normal bottle images taken from the training data received anomaly probabilities of 6.0%, 14.7% and 2.0% and were classified Normal.

6.3 Error Analysis

Figure 5 shows the confusion matrices and Table 8 converts them into error rates. The Random Forest raised 36 false alarms (7.3% of normal images) and missed 60 anomalous images (38.2%). Logistic Regression flagged 226 normal images (45.7%) but missed only 34 anomalies (21.7%), and the Decision Tree flagged 140 normal images (28.3%) and missed 46 anomalies (29.3%). Every model therefore trades false alarms against missed defects.

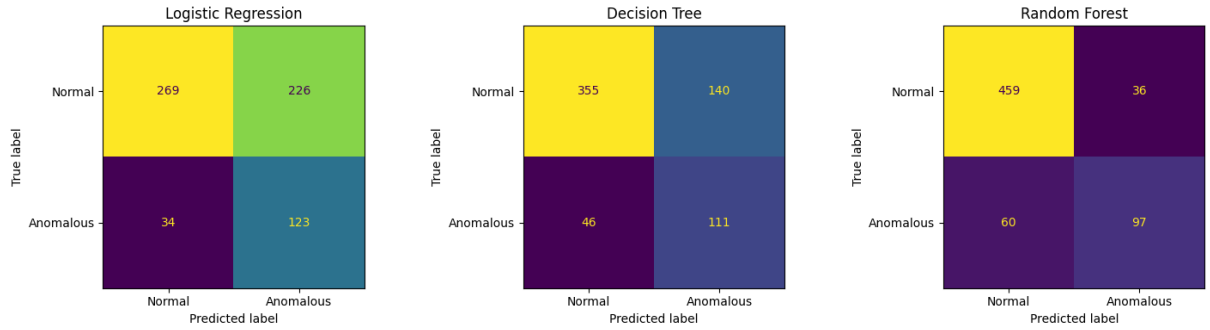


Figure 5. Confusion matrices on the test set (652 images).

Table 8. Error rates on the test set.

Model	False alarms (normal flagged anomalous)	Missed anomalies (anomalous flagged normal)
Random Forest	36 of 495 (7.3%)	60 of 157 (38.2%)
Decision Tree	140 of 495 (28.3%)	46 of 157 (29.3%)
Logistic Regression	226 of 495 (45.7%)	34 of 157 (21.7%)

The missed anomalies are most likely subtle or small defects that barely change the global colour and edge statistics of a 128×128 image; this is a plausible explanation that was **not tested**. A breakdown of errors by category or defect type was not produced because predictions were not stored with image identifiers; it is recommended for the next stage. Errors for leather cannot be studied because that category has no anomalous images.

7 Application Development

7.1 Application Purpose and Users

The intended users are quality-control engineers, assembly-line operators and manufacturing auditors. The application supports a first-pass screening decision: whether a component image looks normal or anomalous and how confident the model is. Uncertain or anomalous results should be passed to a trained human inspector. The user opens the Make Prediction page, selects the object category, uploads a PNG or JPG image (or types the feature values) and presses Predict.

7.2 Application Architecture

The flow is shown in Figure 6. The input is validated (file type and size up to 10 MB, minimum 32×32 pixels, value ranges, valid category); the nine features are extracted from the image exactly as in training; the values are arranged into the 21 training columns in the saved order with the selected category set to 1; the saved Random Forest returns the probability of class 1 (anomalous); and the result, a probability bar and a plain-language explanation are displayed. A warning is shown when the probability lies between 35% and 65%. Each prediction (category, source, three key features, probability and result) is stored in a local SQLite database and shown in a history table; uploaded images are processed in memory and are not stored.

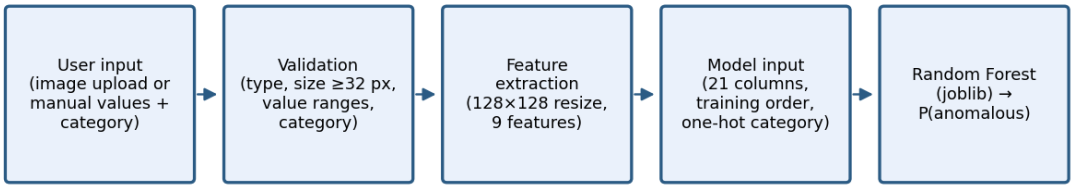


Figure 6. Application architecture.

7.3 Application Features

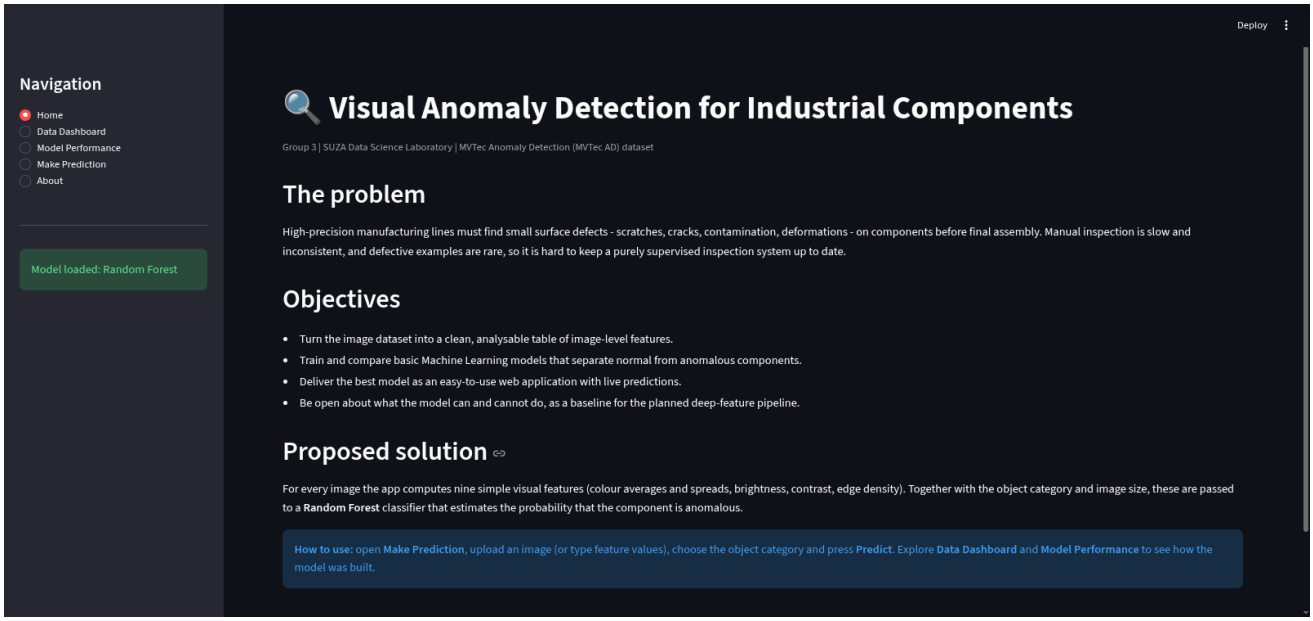
- **Home:** project problem, objectives, proposed solution and short instructions. (Implemented)
- **Data Dashboard:** dataset summary metrics, variable descriptions, data preview and four graphs with interpretation. (Implemented)
- **Model Performance:** evaluation measures, model comparison table and chart, confusion matrices, selected model with justification and top predictors. (Implemented)
- **Make Prediction:** category dropdown, image upload or manual numeric inputs with valid ranges, a Predict button, a clearly explained result with probability and uncertainty warning. (Implemented)
- **About:** intended use, limitations and a responsible-use statement. (Implemented)
- **Enhancements:** database integration (SQLite prediction history with CSV download) and sidebar navigation were implemented; login and role-based access and online deployment were **not** implemented.

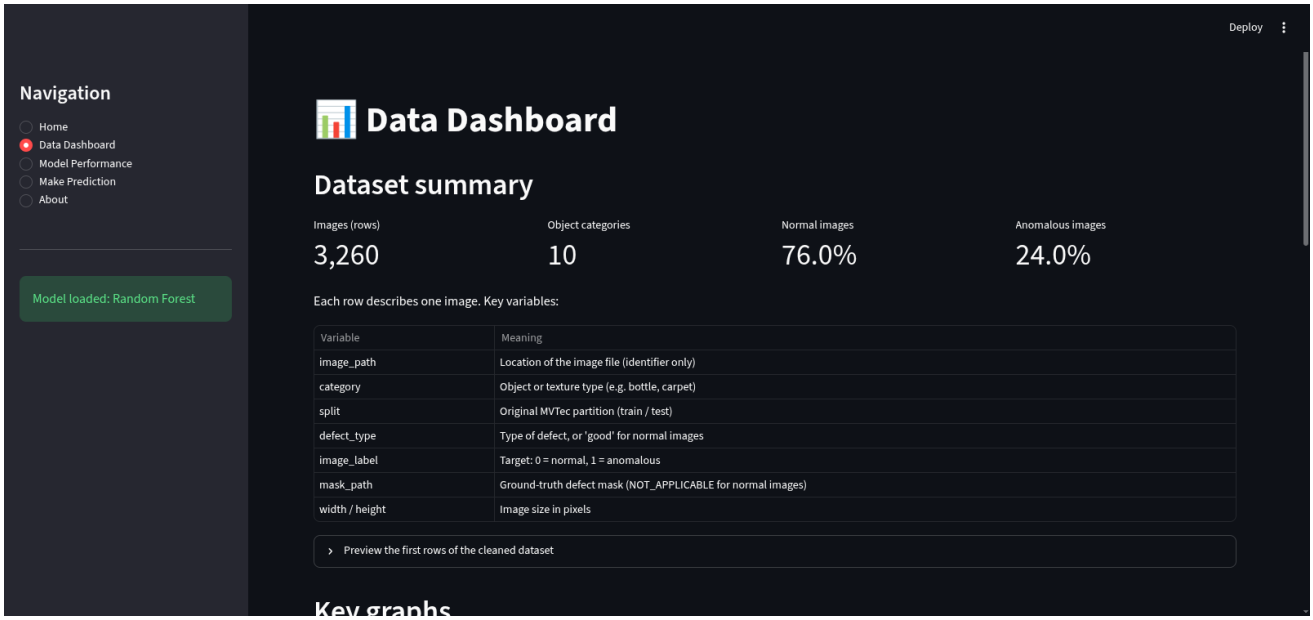
7.4 Model Integration

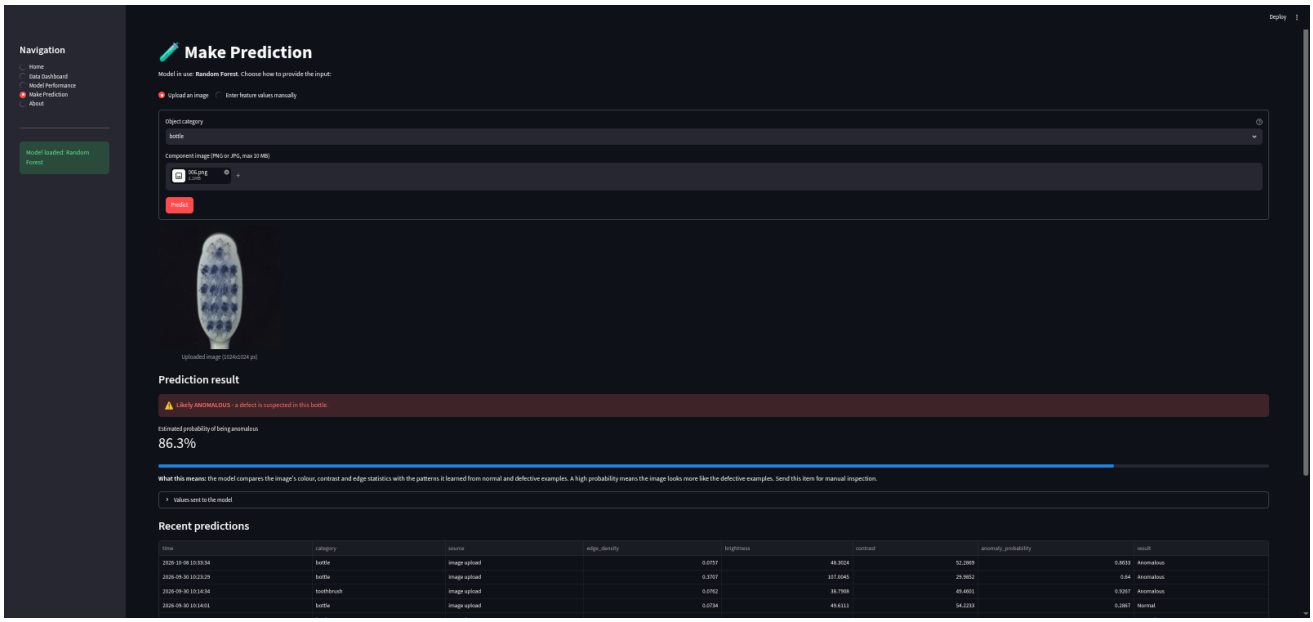
The application loads the saved Random Forest (best_model_random_forest.joblib), the training column list (feature_columns.joblib) and the scaler (feature_scaler.joblib) with joblib, cached once per session; the model is never retrained. The column list guarantees the same variables and order as in training. The scaler is applied only when the saved model is a Logistic Regression, mirroring training (the Random Forest used unscaled inputs). The probability of class 1 is read with predict_proba and compared with a 0.5 threshold. The scikit-learn version is pinned to 1.6.1 in requirements.txt because the model was saved with that version.

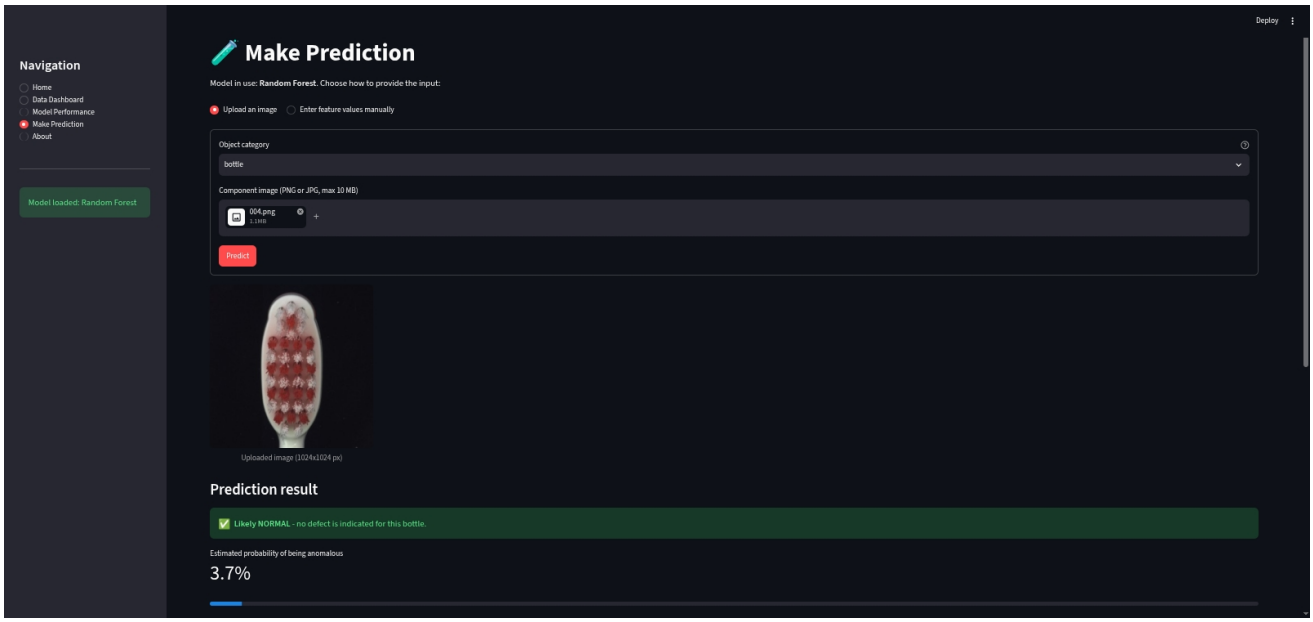
7.5 Screenshots

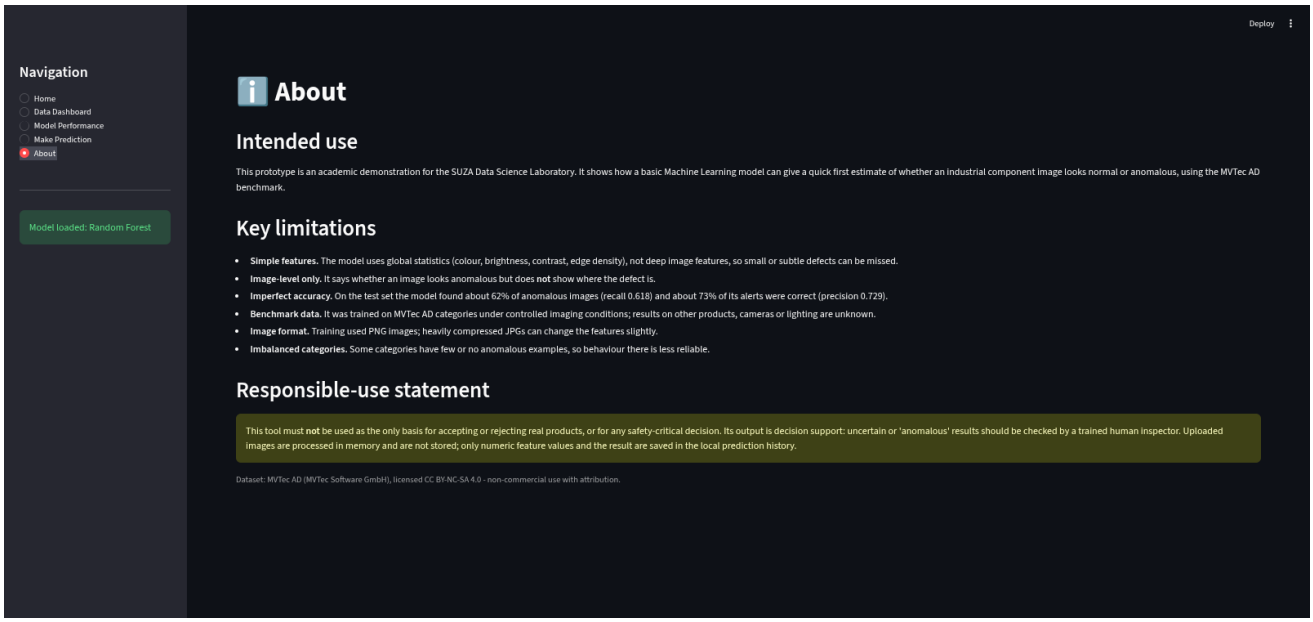
The five main pages and what each allows the user to do are listed below.






Navigation
Home
Data Dashboard
Model Performance
Make Prediction
About
Model loaded: Random Forest
Make Prediction
Model in use: Random Forest. Choose how to provide the input:
Upload an image
Enter feature values manually
Object category
bottle
Component image (PNG or JPG, max 10 MB)
006.png
Predict
Uploaded image (1024x1024 px)
Prediction result
Likely ANOMALOUS - a defect is suspected in this bottle.
Estimated probability of being anomalous
86.3%
What this means: the model compares the image's colour, contrast and edge statistics with the patterns it learned from normal and defective examples. A high probability means the image looks more like the defective examples. Send this item for manual inspection.
Values sent to the model
Recent predictions
time
category
source
edge_density
brightness
contrast
anomaly_probability
result
2025-10-08 10:33:34
bottle
image upload
0.0757
48.3024
52.2869
0.8633
Anomalous
2025-09-30 10:23:29
bottle
image upload
0.3707
107.6045
29.9852
0.64
Anomalous
2025-09-30 10:14:34
toothbrush
image upload
0.0762
38.7988
49.4601
0.9267
Anomalous
2025-09-30 10:14:01
bottle
image upload
0.0734
49.6111
54.2233
0.2867
Normal


Navigation
Home
Data Dashboard
Model Performance
Make Prediction
About
Model loaded: Random Forest
Make Prediction
Model in use: Random Forest. Choose how to provide the input:
Upload an image
Enter feature values manually
Object category
bottle
Component image (PNG or JPG, max 10 MB)
004.png
Predict
Uploaded image (1024x1024 px)
Prediction result
Likely NORMAL - no defect is indicated for this bottle.
Estimated probability of being anomalous
3.7%



8 System Testing and Deployment

8.1 Functional Testing

The application was tested with Streamlit's AppTest framework and function-level checks, using the real Random Forest, column list and scaler. All 15 automated checks passed (Table 9).

Table 9. Functional test cases and results.

Test Case	Expected result	Actual result
Valid input	A prediction, probability bar and explanation are shown	Shown; Pass
Boundary: minimum	Minimum allowed values (size 32 px, statistics 0) are accepted	Prediction produced; Pass
Boundary: maximum	Maximum allowed values (size 10,000 px, means 255, edge density 1.0) are accepted	Prediction produced; Pass
Categories	Each of the 10 categories can be selected and predicted	All 10 worked; Pass
Invalid value	Mean red = 300 is rejected with a message; no prediction	"Mean red must be between 0 and 255 (got 300)"; Pass
Missing value	A blank or missing value is rejected	"Contrast is missing" / "Width is missing"; Pass

Test Case	Expected result	Actual result
Invalid size or category	Width 10 px or empty category is rejected	"Width must be between 32 and 10000 pixels (got 10)"; Pass
Image upload	PNG and JPG are converted to the nine features and the correct size	200×150 test images processed; Pass
Invalid files	Non-image, corrupt and too-small files give a friendly error	Error messages shown, no crash; Pass
Prediction output	Output follows the input; input columns match training order	Probability changed with edge density; 21 columns in training order; Pass
Real-data check	Normal bottle rows from the Assignment 4 run are classified Normal	6.0%, 14.7% and 2.0% anomaly probability; Pass
Missing model files	A clear message instead of a crash	"No model file found" shown; Pass
Navigation	All five pages load without errors	Pass

8.2 Usability Review

No formal usability test with outside users was carried out, so no user feedback is available; this should be done before real use. The interface was designed for non-technical users: every input has a plain label and a valid range, errors state exactly what to correct, the result is given as Normal/Anomalous with a percentage and a probability bar, a warning appears when the model is unsure, and the About page explains the limits. A short test with three to five quality-control staff is recommended.

8.3 Deployment and User Instructions

The application runs **locally**; it has not been deployed online and there is no deployment link. To run it on Linux (the commands on Windows differ only in activating the environment):

Step 1. Unzip mvtec_streamlit_app.zip and open a terminal in the folder.

Step 2. Create and activate an environment: python3 -m venv venv, then source venv/bin/activate.

Step 3. Install the packages: pip install -r requirements.txt.

Step 4. Start the application: streamlit run app.py, then open http://localhost:8501 in a browser.

The README gives the optional steps for deployment on Streamlit Community Cloud. Because that hosting has a temporary file system, the prediction history would reset whenever the app restarts.

9 Ethical Considerations and Limitations

9.1 Ethical and Responsible Use

Privacy. The dataset contains images of industrial parts and no personal data; uploaded images are processed in memory and not stored, and only numeric feature values and the result are saved in the local history. **Sensitive variables.** None are used. **Bias.** The model reflects the benchmark: some categories have more examples than others and leather has no anomalous examples, so performance differs by category and is unknown for leather. **Transparency.** Metrics, limitations and a responsible-use statement are shown inside the application. **Appropriate use.** The tool is for academic demonstration and decision support. **Risks of reliance.** On the test set the selected model missed 38.2% of anomalous images and raised false alarms on 7.3% of normal images; it must not be the only basis for accepting or rejecting real products or for safety-critical decisions. The data licence (CC BY-NC-SA 4.0) allows non-commercial use only.

9.2 Project Limitations

Dataset. MVTec AD is a benchmark photographed under controlled conditions; results do not transfer automatically to other products, cameras or lighting. The working copy has 3,260 images in 10 categories, fewer than the 5,354 images in 15 categories of the full benchmark, and leather has no anomalous images.

Measurement. The features are global statistics of a 128×128 resized image, so small defects can be lost; heavily compressed JPG files can shift the features slightly.

Modelling. The delivered model is a supervised baseline, not the unsupervised, deep-feature detector described in the title. There was no validation set, no hyperparameter tuning, no cross-validation and no ROC-AUC; the model was chosen on the test set from a single split; and errors by category were not analysed.

Application. It is a local prototype without login; it classifies whole images and does not show where a defect is.

Generalisation. Performance on other categories, real production images or new defect types is unknown.

10 Conclusion and Recommendations

10.1 Conclusion

The project addressed automated visual inspection of industrial components using MVTec AD. The data were audited, cleaned and turned into an analysis-ready table of 3,260 images; nine image features were engineered; three supervised models were compared; and the best, a Random Forest (accuracy 0.853, precision 0.729, recall 0.618, F1-score 0.669 on the anomalous class), was delivered in a five-page Streamlit application with validated inputs, an explained result and a prediction history. Table 10 states honestly which objectives were achieved.

Table 10. Achievement of the project objectives.

Objective	Status	Evidence / gap
Specific 1: extract patch-level embeddings with a pretrained CNN backbone	Not achieved	Only nine global, hand-crafted image statistics were computed; no CNN, patch-level features or embeddings were used.
Specific 2: distance-based memory-bank scoring with image decisions and pixel heatmaps	Not achieved	No memory bank or heatmaps; a supervised classifier gives image-level decisions only.
Specific 3: evaluate with Image-AUROC and Pixel-PRO against masks	Not achieved	Accuracy, precision, recall and F1 were used; AUROC, PRO and the masks (783 available) were not used.
General objective: unsupervised detection and localization trained without defective images	Partially achieved	Data preparation, a supervised baseline and a working application exist; the unsupervised, localizing method is still to be built.
Practical: prepare data, compare ML models, deliver an application	Achieved	Cleaned dataset, three compared models, a selected model and a tested Streamlit prototype.

The delivered Random Forest is therefore a useful supervised baseline against which the planned unsupervised PatchCore-style detector can be compared [7].

10.2 Recommendations

Practical recommendations. Use the application only as a screening aid; send anomalous and uncertain (35% to 65%) results for manual inspection; do not use it for safety-critical decisions; and validate it on the factory's own images before any real use.

Technical recommendations for the next stage.

- Implement the unsupervised pipeline: extract patch embeddings from the 2,210 normal training images with a frozen pretrained CNN (for example WideResNet-50), build a memory bank, score test patches by distance, and produce anomaly heatmaps [7].
- Evaluate it with Image-AUROC and Pixel-PRO using the 783 ground-truth masks, and compare it with this baseline.
- Use a validation set or stratified cross-validation, tune hyperparameters, report ROC-AUC, and analyse errors by category and defect type.
- Obtain the missing categories and anomalous leather images, test on real production images, and collect user feedback.
- Add login and roles and deploy the application online if wider use is required.

References

[1] MVTec Software GmbH. "Industrial anomaly detection benchmark dataset: MVTec AD." https://www.mvtec.com/research-teaching/datasets/mvtec-ad

https://github.com/Hades3942/Industrial-Anomaly-detection-app[2] Bergmann, P., Batzner, K., Fauser, M., Sattlegger, D. and Steger, C. (2021). "The MVTec Anomaly Detection Dataset: A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection." International Journal of Computer Vision. https://doi.org/10.1007/s11263-020-01400-4

[3] Kaggle. "MVTec AD" dataset mirror. https://www.kaggle.com/datasets/ipythonx/mvtec-ad

[4] Pedregosa, F. et al. (2011). "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 12, 2825-2830.

[5] Streamlit documentation. https://docs.streamlit.io

[6] Breiman, L. (2001). "Random Forests." Machine Learning, 45(1), 5-32.

[7] Roth, K., Pemula, L., Zepeda, J., Scholkopf, B., Brox, T. and Gehler, P. (2022). "Towards Total Recall in Industrial Anomaly Detection." IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

[8] Bradski, G. (2000). "The OpenCV Library." Dr. Dobb's Journal of Software Tools.

[9] McKinney, W. (2010). "Data Structures for Statistical Computing in Python." Proceedings of the 9th Python in Science Conference.

[10] Hunter, J. D. (2007). "Matplotlib: A 2D Graphics Environment." Computing in Science & Engineering, 9(3), 90-95.

Appendices

Appendix A: Variable dictionary

The full variable dictionary is given in Table 2 (Section 2.3).

Appendix B: Additional tables and model results

Table B1. Per-class precision, recall and F1-score on the test set (support: 495 normal, 157 anomalous).

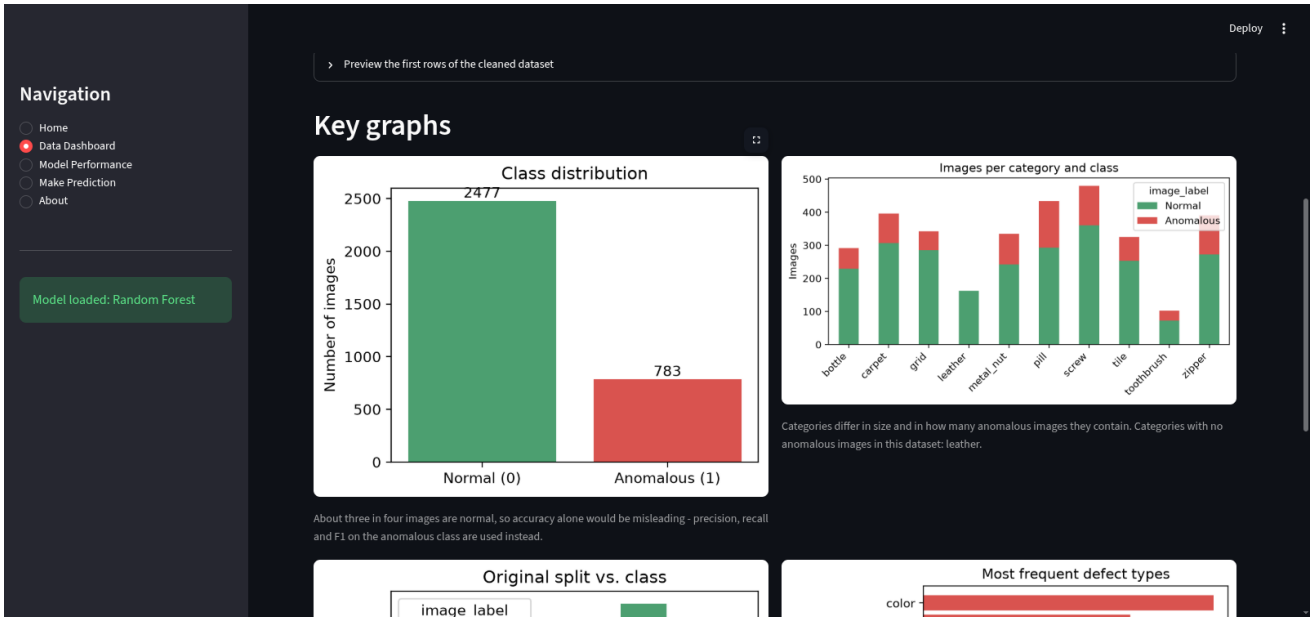
Model	Class	Precision	Recall	F1-score
Logistic Regression	Normal	0.89	0.54	0.67
	Anomalous	0.35	0.78	0.49
Decision Tree	Normal	0.89	0.72	0.79
	Anomalous	0.44	0.71	0.54
Random Forest	Normal	0.88	0.93	0.91
	Anomalous	0.73	0.62	0.67

The ten largest Random Forest feature importances are: edge_density 0.133, std_b 0.119, mean_b 0.118, mean_r 0.113, mean_g 0.105, brightness 0.099, std_g 0.096, std_r 0.093, contrast 0.090 and width 0.006.

Appendix C: Application screenshots and test evidence

Test evidence is summarised in Table 9 (Section 8.1)





Appendix D: Notebooks, source code and project files

- Group3_MVTec_AD_Complete_Project.ipynb: Completed google colab notebook from day1 upto model developments
- mvtec_streamlit_app.zip: app.py, requirements.txt, README.md, models (three .joblib files), data (cleaned_project_dataset.csv) and screenshots folder.

Repository link (Github repo): https://github.com/Hades3942/Industrial-Anomaly-detection-app